

P2P Send Pipe – App-First Architecture & Implementation Plan (v2)

This is an app-first rewrite of the original “WebRTC MVP plan”. It assumes the end user installs apps on each device and uses the product through normal OS affordances (tray/menu bar, drag & drop, share sheet), not dev steps like opening multiple browser windows.

0) Product Definition (What users experience)

Target user

One person with multiple devices (e.g., Mac + Windows PC + laptop), who wants fast “send to myself” transfers.

Primary jobs-to-be-done

- Send a file to another device in 2–3 clicks (or via right-click / share).
- Send a short snippet of text instantly.
- (Optional) Keep clipboard text in sync across devices.

Core UX principles

- ****No account required**** for MVP.
- ****Optional sign-in**** can remove pairing codes in the happy path (“send to my devices”).
- ****Pair once, then instant send**** (trusted peers list).
- ****Local-first****: transfers are P2P; server is only for discovery + signaling.
- ****Visible, safe, reversible****: accept/decline, progress, cancel, clear device list.

User flows

****Install + First run****

1. User installs app on Device A and Device B.
2. App shows “This device” + a “Pair a device” button.

****Pairing****

1. Device A: “Pair a device” → shows QR + 6-digit code (expires in ~5 minutes).
2. Device B: “Pair a device” → scan QR / enter code.
3. Both devices display the same ****safety phrase****; user confirms “Match” on both.
4. Devices appear under “Trusted devices”.

****Account mode (recommended happy path)****

1. User signs in (e.g., Google) on all devices.
2. App shows “My devices” automatically for that account.
3. Sending targets any of the user’s online devices (pairing becomes optional/backup).

****Send****

1. User drags a file onto the tray/menu bar icon or uses “Send...” in app.
2. Picks a trusted device.
3. Receiver gets a system notification + accept/decline.
4. Transfer completes; receiver reveals file in downloads folder.

1) App-Level Architecture (real app, not a demo)

Desktop app (recommended: Tauri)

Modules:

- UI shell: tray/menu bar + small window for device list + send queue
- Device identity: keypair + stable deviceId
- Pairing UX: QR/code + safety phrase verification
- Signaling client: WebSocket presence + pairing + SDP/ICE relay
- WebRTC manager: PeerConnection + DataChannel lifecycle
- Transfer engine: offers, accept/decline, chunking, backpressure, retries
- OS integration:

- Filesystem read/write
- Notifications
- Auto-launch (optional)
- Clipboard access (future)

Signaling server

- WebSocket endpoint
- Device presence tracking
- Pairing session management (short code, TTL, one-time use)
- SDP offer/answer relay
- ICE candidate relay
- Optional auth + “my devices” registry (account-scoped presence)
- Optional LAN discovery (UDP broadcast) to auto-find the signaling server on the same network

Optional future infrastructure

- TURN relay for restricted networks
- “Offline queue” (cloud fallback) if P2P fails (post-MVP)

2) Security + Trust (MVP but app-grade)

Facts:

- WebRTC DataChannels are encrypted (DTLS) end-to-end for payload-in-transit.
- The signaling server can observe metadata (who talks to who, timing, sizes).

MVP security posture:

- Pairing establishes *trust* between device identities (public keys).
- Safety phrase is derived from both public keys as a user verification step.

Next-step hardening (post-MVP, recommended):

- Sign critical signaling messages (offer/answer/ice) with device keys.
- Add “revoke trust” and “pairing history” UI.

3) Wire Protocols

WebSocket messages (signaling)

- `hello` (deviceId, name, publicKeyJwk)
- `presence` snapshot
- `pair.create` → `pair.created` (code, expiresAt)
- `pair.join` → `pair.matched` (peer identity)
- `webrtc.offer/answer/ice` relay (to/from)

DataChannel messages (transfer)

Control JSON:

- `HELLO`
- `FILE\_OFFER`
- `ACCEPT` / `DECLINE`
- `CHUNK\_ACK` (Phase 4)
- `DONE`
- `CANCEL` (Phase 4)

Binary chunks:

- Header: `fileId` + `chunkIndex` + `payloadLength`
- Payload: bytes

4) Productized Phases (end-user checkpoints)

Each phase should be shippable: installable app, minimal configuration, and obvious user value.

Phase 1 – Pairing + Presence (App install experience)

Goal: Users can install on two devices, pair them, and see online status.

Deliverables:

- Installer/package (dev builds acceptable initially)
- Pair a device UI (QR + code)
- Trusted devices list with online/offline
- "Remove device" action

Acceptance criteria:

- Pairing works reliably without requiring restarts.
- Devices persist across app restarts.

Phase 2 – WebRTC Connection (P2P channel)

Goal: Trusted devices can connect and keep a DataChannel open.

Deliverables:

- "Connect" behind the scenes when sending (no manual connect required in final UX)
- Clear error UI if P2P fails (STUN-only)

Acceptance criteria:

- Connection success rate is reasonable on typical home networks.

Phase 3 – File Transfer MVP (Small files)

Goal: Send small files with accept/decline + progress.

Deliverables:

- Send file UX (tray action + window picker)
- Receiver notifications + accept/decline
- Progress + completion toast
- Default save location (Downloads) + "Reveal in Finder/Explorer"

Acceptance criteria:

- A 5-20MB file transfers correctly and is checksum-valid.

Phase 4 – Reliability (Real-world behavior)

Goal: Robust transfers.

Deliverables:

- Backpressure handling (bufferedAmount)
- Chunk acks + retransmit window (or simpler: sequential send + ack)
- Cancel / timeout handling
- Resume is optional; can be future

Acceptance criteria:

- Large files complete without UI freezing and without corrupt output.

Phase 5 – Clipboard Sync (Text)

Goal: Optional text clipboard sync with loop prevention.

Deliverables:

- Toggle per device and global
- Change detection + debounce
- Loop prevention (nonce + origin device)

Acceptance criteria:

- No "clipboard ping-pong" loops.

5) Testing Strategy (app-focused)

Unit:

- Protocol encoding/decoding
- Chunk framing + ordering
- Retry/cancel state machine

Integration (local):

- Pairing flow (two app instances)
- Signaling reconnect behavior
- WebRTC offer/answer exchange
- File transfer correctness + checksum

Manual:

- Send file between two physical machines on different networks

- Sleep/wake behavior
- Firewall/NAT edge cases

6) Observability (MVP)

App-side logs per transfer:

- peerId, fileId, size, start/end time
- throughput
- failures + reason codes

Server-side:

- device presence counts
- pairing code creation/join stats
- signaling relay failures

7) Recommended Tech Stack (practical)

Desktop app:

- Tauri (Rust shell + web UI) or Electron

Server:

- Node.js WebSocket server (hosted minimal)

WebRTC:

- Native WebRTC (browser engine if web UI; or platform bindings if needed later)

Storage:

- Local filesystem only for MVP (Downloads + configurable)

8) Open Decisions (pick before polishing UI)

- Packaging target: macOS only first, or macOS + Windows?
- Is the app “always running” (tray) or “launch on demand”?
- Default save directory + rules (overwrite, rename, prompt)
- Notifications permissions and fallback UI